MÓDULO DIW · DAW — UNIDAD DIDÁCTICA 5

Arquitectura CSS (BEM) y Utility-First con Tailwind CSS

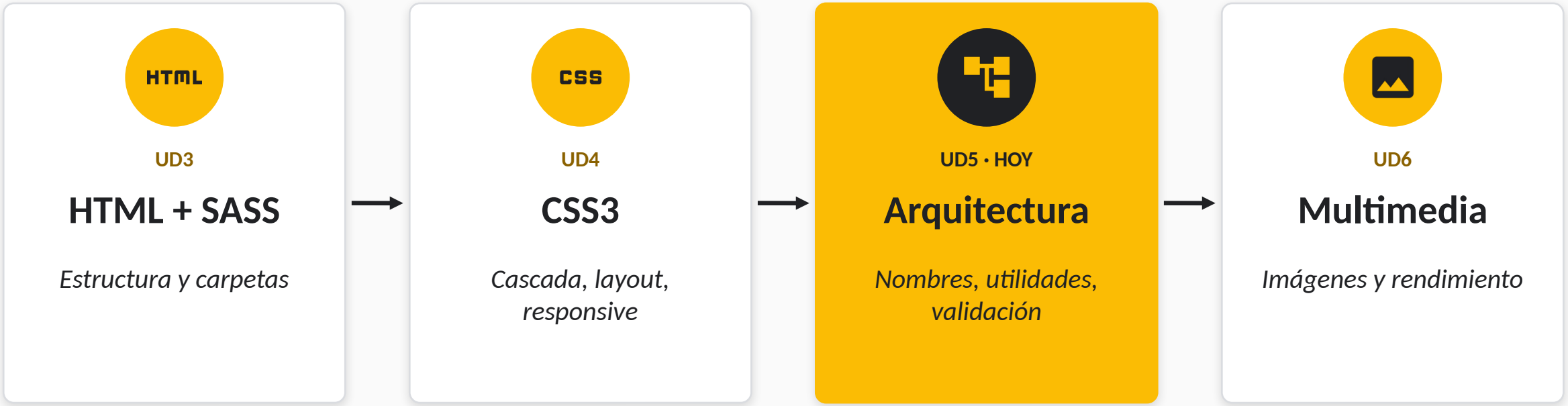
RA2 · Crea interfaces web homogéneas definiendo y aplicando estilos

- Crear clases de estilo con una convención que entienda todo el equipo (BEM)
- Componer interfaces con clases de utilidad (Tailwind CSS)
- Validar el CSS con herramientas y mantener viva la guía de estilo



DÓNDE ESTAMOS

AURIA ya tiene piel: hoy ordenamos el armario



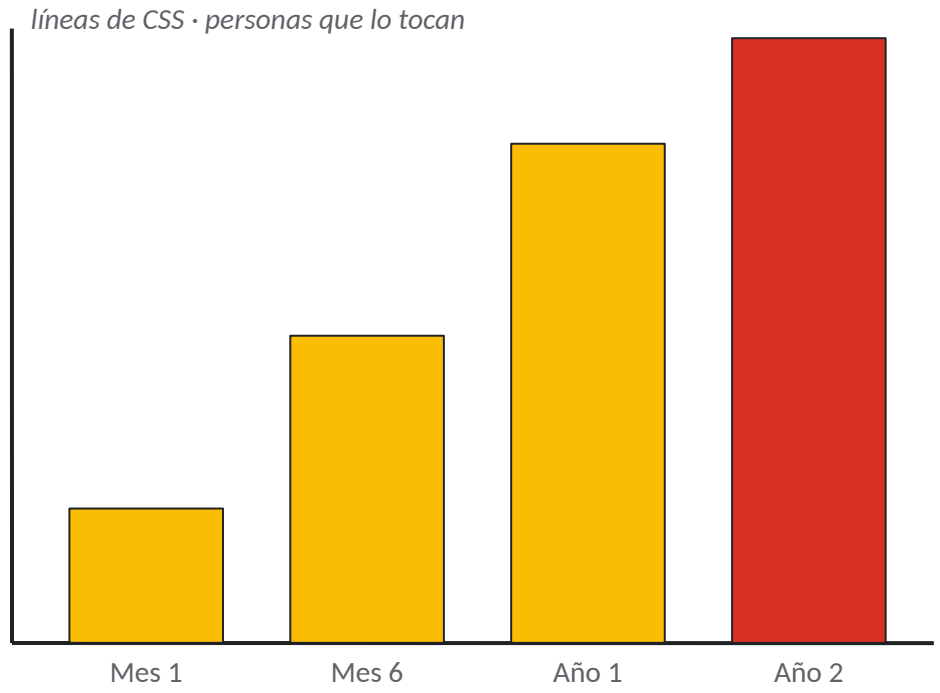
La pregunta de la unidad: ¿cómo escribimos CSS que otra persona pueda tocar sin miedo dentro de seis meses?

Con esta unidad se cierra el RA2. En la UD6 empieza el RA3: recursos multimedia.

CONTEXTO PROFESIONAL

El CSS se escribe una vez y se lee cien

Un proyecto real con el tiempo



Varias personas, un mismo CSS

Lo que tú escribes hoy lo modifica otra persona mañana

Se lee más de lo que se escribe

Un nombre claro ahorra horas de buscar «qué hace esta clase»

Miedo a borrar

Sin convención nadie sabe si una regla se usa: el CSS solo crece

Revisión automática

En las empresas el CSS pasa un linter antes de entrar al proyecto

Diseño y código sincronizados

La guía de estilo es el contrato entre diseño y desarrollo

DATO CLAVE — El problema del CSS no es escribirlo: es mantenerlo cuando crece y cuando lo toca más de una persona.

HOJA DE RUTA

Cuatro bloques, de la convención a la guía de estilo

1 · El problema

Nombres sin sentido, especificidad que sube, miedo a borrar

2 · BEM

Bloque, elemento y modificador: nombres que explican la estructura

3 · Tailwind CSS

Utility-first: componer con clases pequeñas y tokens en @theme

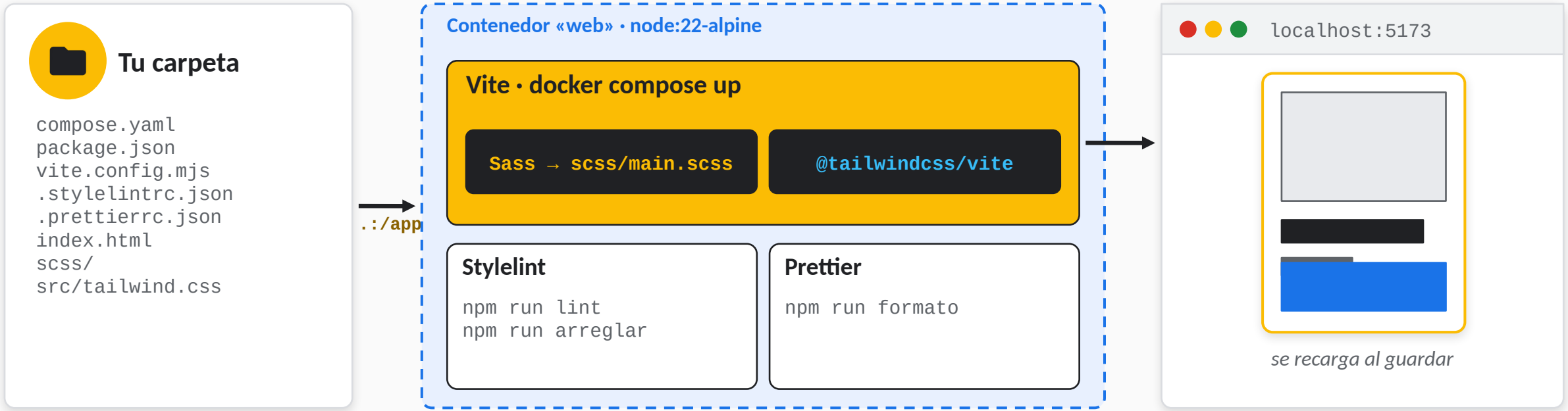
4 · Validar y documentar

W3C, Stylelint, Prettier y la guía de estilo (RA2.h, RA2.i)

Laboratorio: la tarjeta de producto de AURIA, refactorizada en BEM con SASS, la misma tarjeta en Tailwind, Stylelint vigilando los nombres y la guía de estilo al día.

CLAVE — BEM y Tailwind no son rivales que haya que elegir de por vida: son dos respuestas al mismo problema.

Dónde se trabaja: Docker + Vite, como en la UD3



terminal · dentro de la carpeta del proyecto

```
docker compose run --rm web npm install      # la primera vez (con internet): descarga las herramientas
docker compose up                            # Vite en marcha: la página en http://localhost:5173
docker compose run --rm web npm run lint     # cualquier orden de package.json: lint, arreglar, formato…
```

CLAVE — En tu ordenador solo hay Docker. Node, Vite, Sass, Tailwind, Stylelint y Prettier viven en el contenedor.



BLOQUE 1

El problema: CSS que nadie toca

Por qué un proyecto sin convención acaba siendo imposible de mantener

BLOQUE 1 · EL PROBLEMA

Tres síntomas de un CSS sin convención

styles.css (año 2)

```
.card { … }
.card2 { … }
.btn-2 { … }
.rojo { color: red; }
.nuevo-btn-ok { … }

#main .card .btn:hover { … }
div.productos ul li a span { … }

.card .title { … }
.modal .title { … }  /* ¿cuál gana? */
```

Nombres sin significado

.card2, .btn-2, .rojo: ¿qué son? ¿dónde se usan? ¿y si .rojo pasa a ser azul?

Especificidad que sube

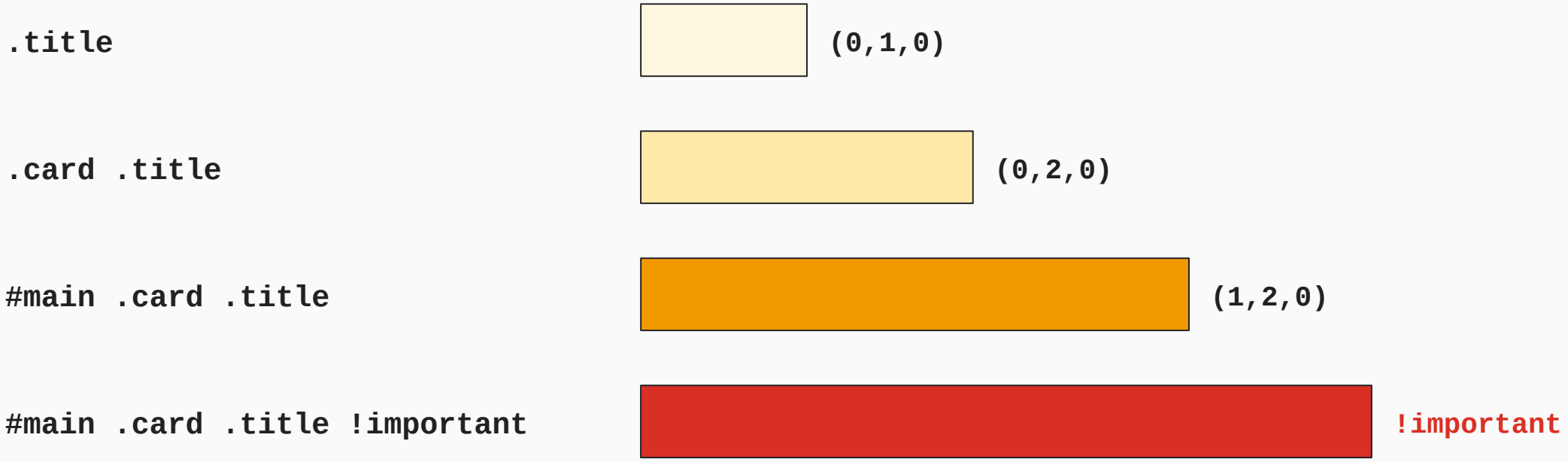
Selectores largos y anidados que solo se pueden vencer con otro más largo… o con !important (UD4)

Estilos que se pisan

.card .title y .modal .title dependen del sitio: mueves el HTML y cambia el aspecto

RESULTADO — Nadie se atreve a borrar nada: se añade otra regla «por si acaso» y el fichero solo crece.

Recordad la UD4: la especificidad es una escalada



La idea de toda la unidad: si TODOS los selectores pesan lo mismo, (0,1,0), nadie tiene que pelear: decide el orden y el nombre.

CLAVE — La especificidad baja y plana no es una manía: es lo que permite cambiar una pieza sin romper otra.

Dos respuestas al mismo problema

CSS sin convención

BEM

Semántica y estructura

Nombras cada pieza según lo que ES dentro de su componente y escribes su CSS una vez.

```
.card__title
```

Utility-first (Tailwind)

Composición atómica

Cada clase hace UNA cosa. Montas el aspecto combinándolas en el HTML.

```
text-xl font-semibold
```

OTRAS — Existen más arquitecturas (OOCSS, SMACSS, ITCSS…). BEM y Tailwind son las más extendidas y las que veremos.

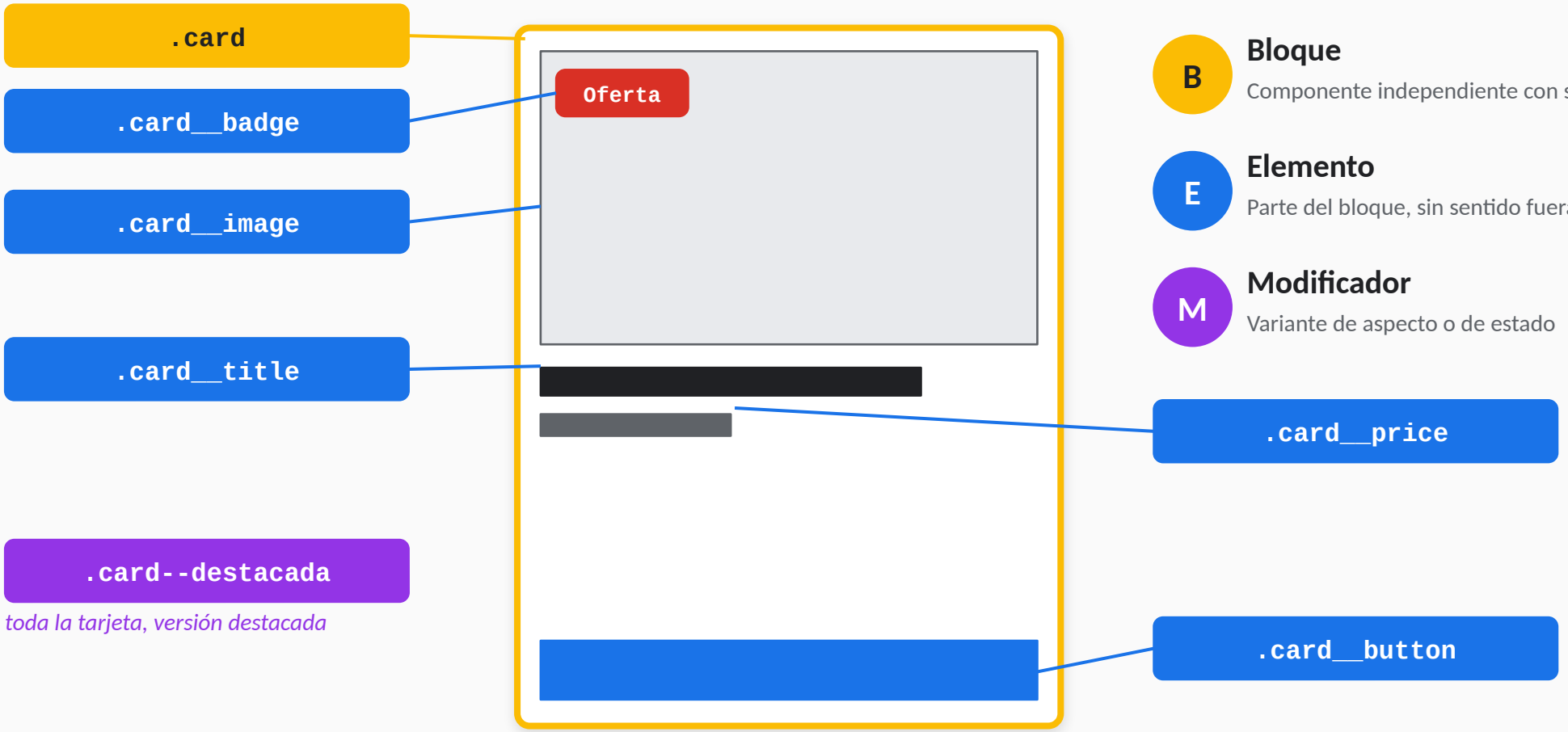


BLOQUE 2

BEM: Bloque, Elemento, Modificador

Una convención de nombres que explica la estructura del componente

Tres piezas: bloque, elemento y modificador



B **Bloque**
Componente independiente con sentido propio

E **Elemento**
Parte del bloque, sin sentido fuera de él

M **Modificador**
Variante de aspecto o de estado

ORIGEN — BEM lo creó Yandex (el buscador ruso) hacia 2009 para que equipos grandes compartieran el mismo CSS.

La sintaxis: dos guiones bajos y dos guiones



Se lee: «el botón de la tarjeta, en su variante principal»

Patrón	Ejemplo	Significado
`.bloque`	.card	El componente
`.bloque__elemento`	.card__title	Una parte del componente
`.bloque--modificador`	.card--destacada	Variante del componente entero
`.bloque__elemento--modificador`	.card__button--primary	Variante de una parte

NOMBRES — En minúsculas y con guion simple dentro de cada palabra compuesta: .product-card__add-button--disabled

La tarjeta de AURIA, con nombres BEM

index.html

```html
<article class="card card--destacada">
  <span class="card__badge badge">Oferta</span>
  <img class="card__image" src="auria-pro.webp"
       alt="Auriculares AURIA Pro">
  <h3 class="card__title">AURIA Pro</h3>
  <p class="card__price">199 €</p>
  <a class="card__button btn" href="/carrito">
    Añadir al carrito
  </a>
</article>
```

1 Modificador + base

class="card card--destacada": el modificador SIEMPRE acompaña a la clase base, no la sustituye

2 Se lee en el HTML

Sin abrir el CSS ya sabes qué pieza es cada cosa y a qué componente pertenece

3 Dos bloques en un nodo

card__badge badge y card__button btn: pieza de la tarjeta Y bloque propio. Es la mezcla (16 y 19)

OJO — BEM no cambia qué etiqueta usas: article, h3, p y a siguen siendo las de la UD3. Solo ordena los nombres de las clases.

BEM y SASS encajan: el & de la UD3

components/_card.scss

```scss
.card {
  display: flex;
  flex-direction: column;
  gap: $space-md;

  &--destacada {
    border: 2px solid $color-brand-primary;
  }
  &__title { font-size: 1.25rem; }
  &__price {
    color: $color-brand-primary;
    font-weight: 700;
  }
}
```

Vite compila



CSS que sirve Vite (F12 › Sources)

```css
.card { display: flex; … }
.card--destacada { border: … }
.card__title { font-size: 1.25rem; }
.card__price {
  color: #1e88e5;
  font-weight: 700;
}
```

Todo el CSS de la tarjeta en un fichero, y en el resultado todas las reglas son de UNA sola clase.



OJO — &__title genera .card__title (una clase), no .card .title (dos). Por eso BEM + SASS no aumenta la especificidad.

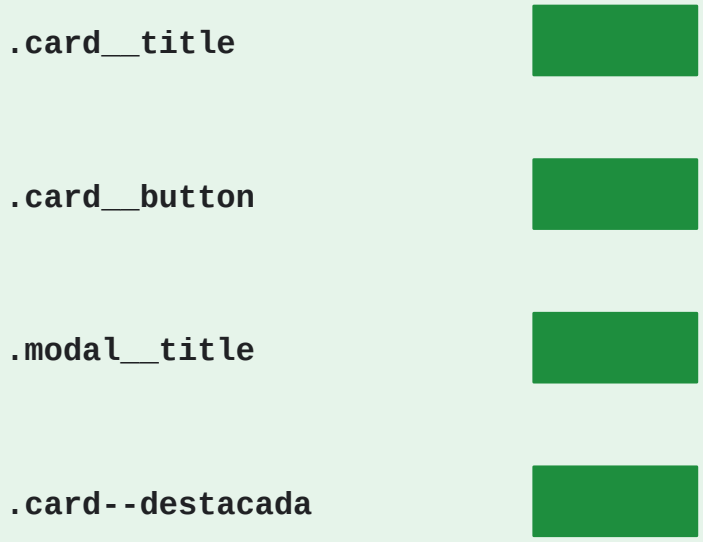
Especificidad plana: todas las reglas pesan (0,1,0)

Sin convención



Un id o un selector anidado pesan más que cualquier clase sola: para cambiarlos hay que escribir algo aún más pesado.

Con BEM



Todas iguales: si dos reglas chocan, decide el orden. Cambiar una pieza no obliga a pelear con otra.

CLAVE — Esta es la alternativa al !important de la UD4: no ganar subiendo el peso, sino que nadie tenga que ganar.

Cuatro reglas que evitan el 90 % de los errores

Sin nietos en el nombre

✗ `.card__body__title`

✓ `.card__title`

El nombre refleja el bloque, no el árbol del HTML

El modificador no va solo

✗ `class="card--destacada"`

✓ `class="card card--destacada"`

El modificador solo trae lo que cambia

Sin selectores de etiqueta

✗ `.card h3 { … }`

✓ `.card__title { … }`

Si cambias h3 por h2, el estilo no se pierde

Bloque dentro de bloque

✗ `.card .btn { … }`

✓ `class="card__button btn"`

Mezcla: el botón es pieza de la tarjeta Y un bloque propio

El modificador trae SOLO lo que cambia

Repite la base

components/_card.scss

```scss
.card {
  display: flex;
  flex-direction: column;
  padding: $space-md;
  border: 1px solid $color-border;

  &--destacada {
    display: flex;            // repetido
    flex-direction: column;   // repetido
    padding: $space-md;       // repetido
    border-color: $color-brand-primary;
  }
}
```

Si mañana cambia el padding de .card, la destacada se queda con el viejo.

Solo lo que cambia

components/_card.scss

```scss
.card {
  display: flex;
  flex-direction: column;
  padding: $space-md;
  border: 1px solid $color-border;

  &--destacada {
    border-color: $color-brand-primary;
  }
}
```

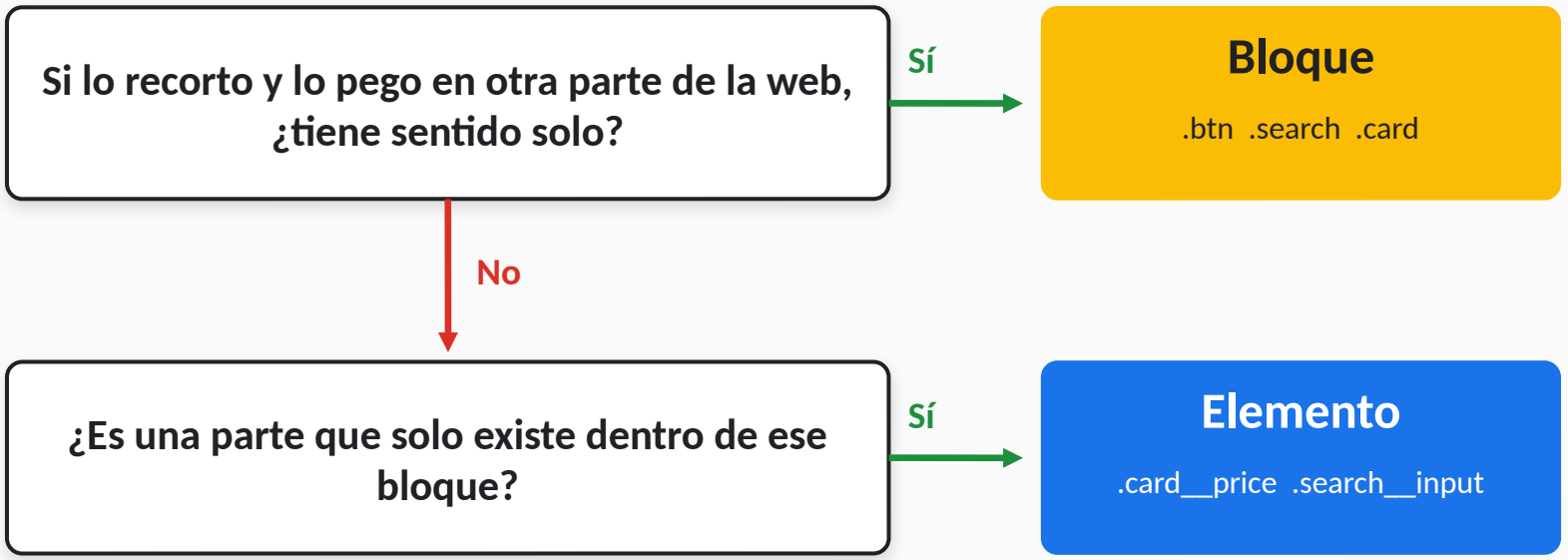
Lo común vive en un sitio. La destacada hereda cualquier cambio de .card.

```html
<article class="card card--destacada">
```

card trae lo común · card--destacada, solo la diferencia (el borde)

PRUEBA — Quita card--destacada del HTML: la tarjeta sigue entera y solo pierde lo que la hacía distinta. Si se rompe, el modificador traía de más.

¿Bloque o elemento? Una pregunta lo decide



Ejemplos de AURIA

Botón «Añadir»: bloque .btn (se usa en toda la web)

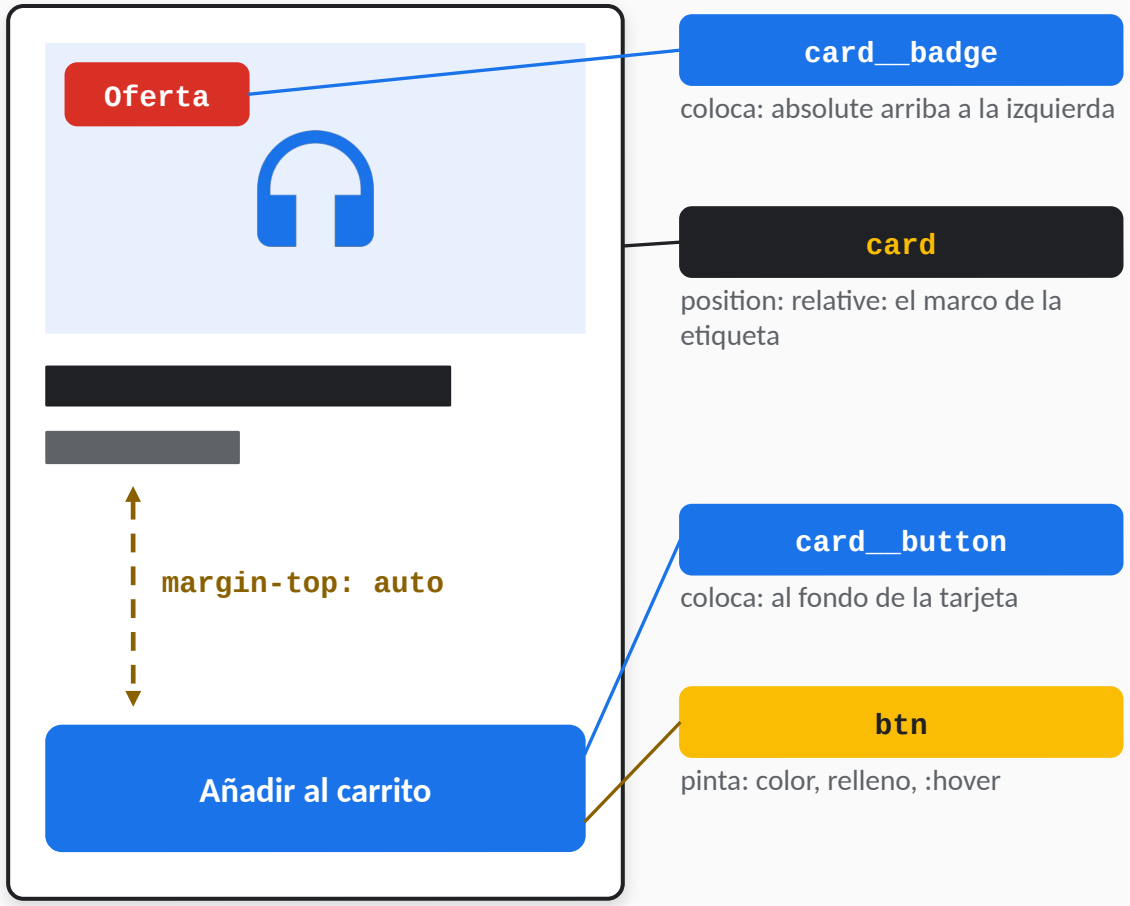
Precio: elemento .card__price

Valoración con estrellas: bloque .rating (aparece también en reseñas)

Etiqueta «Oferta»: elemento .card__badge, o bloque .badge si se reutiliza

CLAVE — No hay una respuesta única: lo importante es que el equipo lo decida y lo escriba en la guía de estilo.

La mezcla: el elemento coloca, el bloque pinta



Clase	Pone (colocar)	Nunca pone (pintar)
`card__button`	margin · margin-top: auto · align-self	background · color · padding · border · font
`btn`	background · color · padding · border · :hover	margin: no sabe dónde lo van a poner
`card__badge`	position: absolute · top · left	colores: los pone el bloque badge

index.html

```html
<article class="card card--destacada">
  <span class="card__badge badge">Oferta</span>
  …
  <a class="card__button btn" href="/carrito">
    Añadir al carrito</a>
</article>
```

POR QUÉ — Si la tarjeta pintara su botón (.card .btn), habría dos botones distintos según dónde estén. Cada clase hace un trabajo.

El único selector de dos clases: .bloque--mod .bloque__elem



Una sola clase en el <article> y la tarjeta entera sabe que está agotada

components/_card.scss

```scss
.card {
  &__image { width: 100%; }

  // compila a .card--agotada .card__image
  &--agotada &__image {
    filter: grayscale(1);
  }
}
```

✅ **`.card--agotada .card__image`**

El que sabe que está agotada es la TARJETA: una clase en el bloque cambia la imagen, el precio y el botón a la vez

❌ **`.card__image--agotada`**

Habría que ponerlo en cada elemento (imagen, precio, botón…). Si se te olvida uno, la tarjeta queda a medias

💡 **LÍMITE —** Pesa (0,2,0), pero solo dentro del componente. Es la única excepción: modificador del bloque + elemento del MISMO bloque.

Estados: modificador, pseudoclase o atributo ARIA

Pseudoclases

Estados que el navegador ya conoce

```
&:hover
```

```
&:focus-visible
```

```
&:disabled
```

El ratón encima, el foco del teclado, el botón deshabilitado

Modificadores

Variantes que decides tú

```
.card--destacada
```

```
.btn--secondary
```

```
.alert--error
```

Cambian el aspecto de forma permanente o según el contenido

Atributos ARIA

Estados que también debe oír el lector

```
[aria-expanded="true"]
```

```
[aria-current="page"]
```

```
[aria-invalid="true"]
```

El CSS y la accesibilidad leen el mismo dato (UD9)

REGLA — Si el estado ya existe en HTML o ARIA, úsalo para el CSS en vez de inventar una clase: así no se desincronizan.

Refactor: el menú heredado de AURIA, a BEM y ARIA

_menu.scss · la web antigua

```scss
#menu {
  ul li a { color: $color-on-brand !important; }
  ul li a.activo { font-weight: 700; }
}
#menu.abierto ul { display: block; }
```

(1,1,3) #menu ul li a.activo: para cambiarlo, algo aún más pesado. La escalada

!important Gana a todo. El siguiente tendrá que poner otro

ul li a Si el menú pasa a <ol> o los enlaces a botones, se pierde el estilo

.activo .abierto El lector de pantalla no los oye, y si el JS deja de poner .abierto, el menú no se abre

_menu.scss · rehecho

```scss
.menu {
  &__lista { display: none; }
  &__enlace { color: $color-on-brand; }
  &__enlace[aria-current="page"] {
    font-weight: 700;
  }
  &__boton[aria-expanded="true"] + &__lista {
    display: block;
  }
}
```

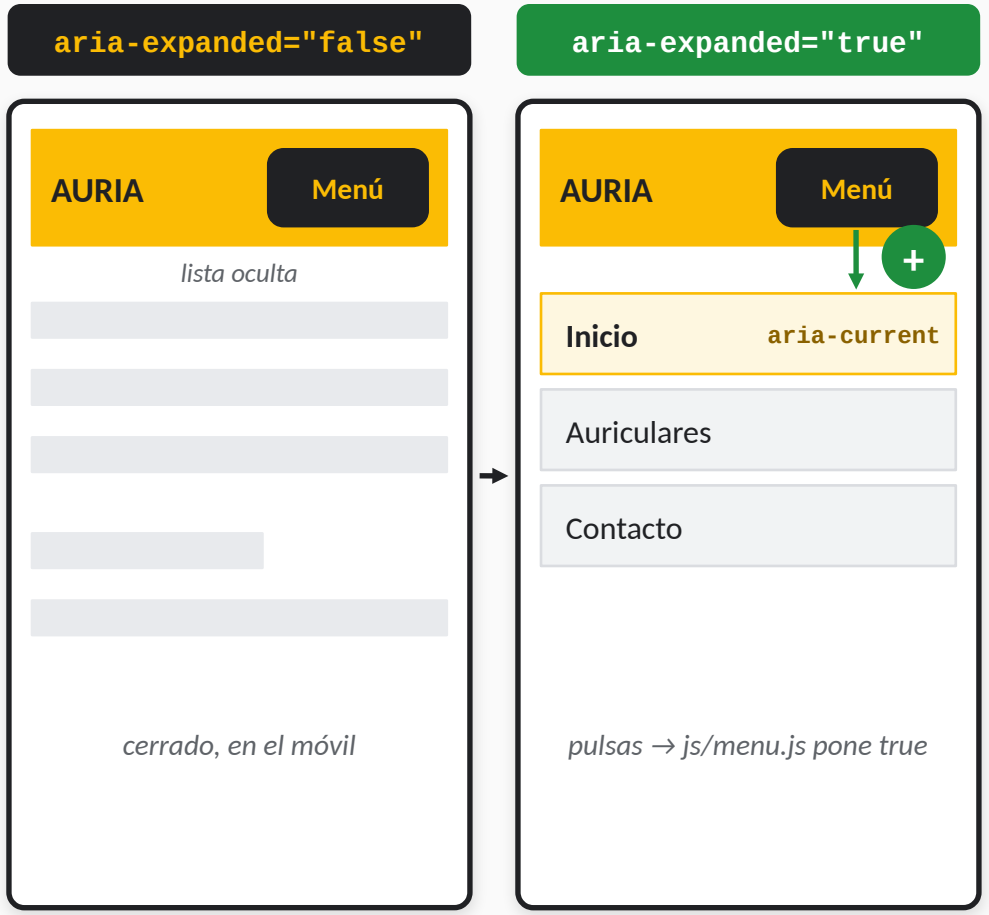
Sin #id, sin !important, sin etiquetas: solo clases BEM y atributos

El enlace actual: de (1,1,3) a (0,2,0), y ahora también lo oye el lector

Los estados salen de ARIA: aria-current y aria-expanded

PASOS — 1 · clases BEM en el HTML 2 · los estados, a ARIA 3 · fuera #id, !important y etiquetas 4 · mobile-first

El menú se abre con aria-expanded y el combinador +



index.html

```html
<nav class="menu">
  <button class="menu__boton" aria-expanded="false"
          aria-controls="menu-lista">Menú</button>
  <ul class="menu__lista" id="menu-lista">
    <li><a class="menu__enlace" href="/"
           aria-current="page">Inicio</a></li>
    …
  </ul>
```

+ — Hermano siguiente: «la lista que va JUSTO después del botón abierto». Por eso el <ul> va pegado al <button>

js/menu.js — Al pulsar solo cambia aria-expanded. El CSS lee ese dato: la pantalla y el lector dicen lo mismo

id="menu-lista" — Se queda: lo usa aria-controls. En el HTML los id valen; lo que no se hace es estilar con ellos

MOBILE-FIRST — La lista, display: none en el móvil; desde $bp-md, @include desde($bp-md) { display: flex; } y el botón se oculta.

Ventajas y límites de BEM

Ventajas

- Nombres que se explican solos en el HTML
- Especificidad plana y predecible
- Un componente = un fichero SASS
- Se puede borrar un bloque entero sin miedo
- Encaja con los componentes de la UD7 (Atomic Design)

Límites

- Nombres largos: .card__button--primary
- Exige disciplina: un solo .card .title y se rompe
- Hay que decidir qué es bloque y qué es elemento
- No guarda los valores: necesita los tokens de la UD2

CLAVE — Los nombres largos son el precio de la claridad: se escriben una vez y se leen cien.



BLOQUE 3

Utility-first con Tailwind CSS

Componer el aspecto con clases pequeñas que hacen una sola cosa

Cambio de paradigma: del nombre a la composición

BEM

1. Pones un nombre: .card__button
2. Escribes su CSS en _card.scss
3. Lo usas en el HTML

HTML limpio · la lógica visual está en el CSS

Utility-first

1. No escribes CSS nuevo
2. Combinas clases ya hechas:
   bg-blue-600 text-white px-4 py-2
3. El aspecto se lee en el HTML

CSS mínimo · la lógica visual está en el HTML

Una clase de utilidad = UNA declaración: `flex → display: flex` · `p-4 → padding: 1rem`

HISTORIA — Tailwind CSS nació en 2017 (Adam Wathan). La versión actual, la 4, se configura desde el propio CSS.

La misma tarjeta de AURIA, en Tailwind

index.html

```html
<article class="relative flex flex-col gap-4 rounded-lg
                border-2 border-brand-primary p-4">
  <span class="absolute top-2 left-2 rounded bg-red-600
               px-2 text-sm text-white">Oferta</span>
  <img class="rounded-md" src="auria-pro.webp"
       alt="Auriculares AURIA Pro">
  <h3 class="text-xl font-semibold">AURIA Pro</h3>
  <p class="font-bold text-brand-primary">199 €</p>
  <a class="rounded-md bg-brand-primary px-4 py-2
            text-white hover:bg-brand-dark
            focus-visible:outline-2" href="/carrito">
    Añadir al carrito</a>
</article>
```



Mismo resultado visual que la versión BEM

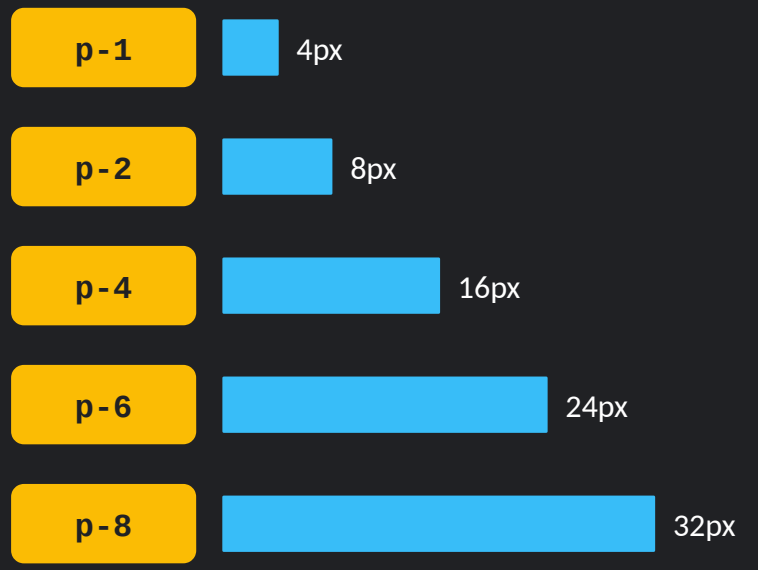
OBSERVA — No hay ni una línea de CSS propio: todo el aspecto está en el atributo class.

Las clases son CSS con nombre corto

CSS	Tailwind
display: flex	`flex`
flex-direction: column	`flex-col`
justify-content: space-between	`justify-between`
gap: 0.5rem (8px)	`gap-2`
padding: 1rem (16px)	`p-4`
padding-inline: 1rem	`px-4`
border-radius: 0.5rem	`rounded-lg`
font-weight: 600	`font-semibold`
display: grid; grid-template-columns: repeat(3, …)	`grid grid-cols-3`

La escala de espaciado

Cada número son 0,25rem (4px):



TRUCO — No hay que memorizarlas: la extensión «Tailwind CSS IntelliSense» de VS Code autocompleta y enseña el CSS de cada clase.

Variantes: estados y responsive con un prefijo

Prefijo	Ejemplo	Equivale a
hover:	`hover:bg-blue-700`	&:hover de SASS
focus-visible:	`focus-visible:outline-2`	&:focus-visible
md:	`md:flex-row`	@media (min-width: 48rem)
lg:	`lg:grid-cols-3`	@media (min-width: 64rem)
dark:	`dark:bg-gray-900`	@media (prefers-color-scheme: dark)

Mobile-first, como en la UD4

```
class="flex-col md:flex-row"
```

- Sin prefijo: se aplica SIEMPRE (la base, el móvil)
- md: añade a partir de 768 px
- Se combinan: md:hover:bg-blue-700

sm 640 · md 768 · lg 1024 · xl 1280 · 2xl 1536

CLAVE — md: significa «a partir de md», no «solo en tablet». Es exactamente el min-width de la UD4.

Tailwind v4 en el proyecto: el plugin de Vite

vite.config.mjs

```
import { defineConfig } from "vite";
import tailwindcss from "@tailwindcss/vite";

export default defineConfig({
  plugins: [tailwindcss()],
});
```

src/tailwind.css

```
@import "tailwindcss";
```

tailwind.html

```
<link rel="stylesheet" href="/src/tailwind.css">
```

1 Ya viene en package.json

tailwindcss y @tailwindcss/vite. Se descargan con el npm install de la primera vez, dentro de Docker

2 El plugin, en vite.config.mjs

Vite ya compilaba el SASS (UD3); con el plugin también compila Tailwind

3 Una línea en src/tailwind.css

@import "tailwindcss"; trae todo Tailwind. Debajo irá tu @theme

4 Enlazar y arrancar

<link> al .css de src/ y docker compose up: guardas y la página se recarga

OJO — Una página con Tailwind NO carga el SASS: así las dos arquitecturas no se mezclan. Nada se instala en tu ordenador.

Los tokens de la UD2 entran por @theme

src/tailwind.css

```css
@import "tailwindcss";

@theme {
  --color-brand-primary: #1e88e5;
  --color-brand-dark: #1565c0;
  --spacing-md: 1rem;
  --radius-card: 0.5rem;
  --font-display: "Inter", sans-serif;
}
```

Variable en @theme	Clases que crea
`--color-brand-primary`	bg-brand-primary · text-… · border-…
`--spacing-md`	p-md · m-md · gap-md
`--radius-card`	rounded-card
`--font-display`	font-display
`--hueco (sin prefijo)`	ninguna: solo var(--hueco)

El PREFIJO decide qué clases salen: --color-, --spacing-, --radius-, --font-, --text-, --breakpoint-. Tailwind no sustituye a los tokens: los implementa. El nombre de la UD2 pasa a ser una clase, y sigue existiendo como variable CSS: var(--color-brand-primary).

VERSIONES — En Tailwind 3 esto se hacía en tailwind.config.js. Lo veréis en muchos tutoriales: la idea es la misma.

Solo genera las clases que usas



Tailwind tiene miles de clases posibles, pero tu CSS final solo trae las que encuentra en tu código: suele quedar en pocos KB.

La trampa

Tailwind busca texto, no ejecuta tu código. Una clase montada a trozos no existe para él:

```
`bg-${color}-600`   ✗
```

Escribe siempre el nombre completo:

```
color === 'rojo' ? 'bg-red-600'
                 : 'bg-blue-600'   ✓
```

CLAVE — Si una clase «no hace nada», lo primero: ¿aparece escrita entera en algún fichero que Tailwind lee?

Salidas de emergencia: valores arbitrarios y @apply

Valores arbitrarios

```
w-[327px]   top-[13px]   bg-[#ff5a1f]
```

Escriben un valor exacto entre corchetes. Útiles para un caso puntual… y peligrosos: son los valores mágicos de la UD2 otra vez.

Si lo repites, crea un token en @theme

@apply

```
.btn {
  @apply rounded-md bg-brand-primary
         px-4 py-2 text-white;
}
```

Junta varias utilidades en una clase propia. Sirve cuando no tienes componentes (HTML repetido a mano).

Con moderación: si todo es @apply, vuelves a escribir CSS

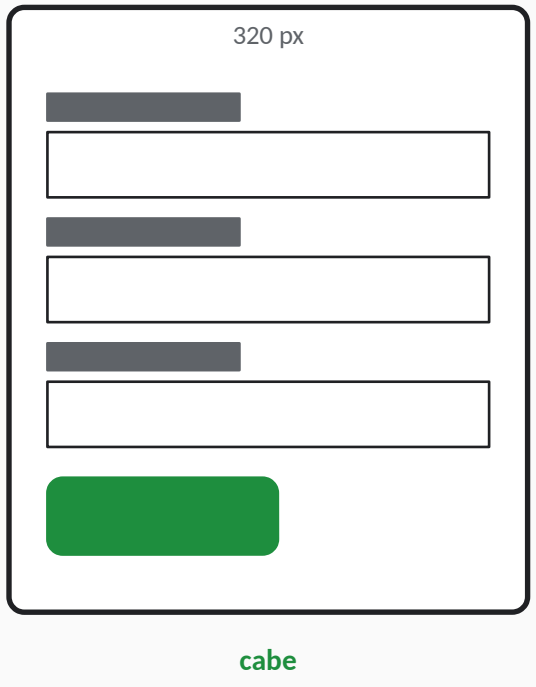
CLAVE — Si en tu HTML hay muchos corchetes, no estás usando el sistema de diseño: lo estás saltando.

Dos trampas: la rejilla que desborda y los tokens en dos sitios

`grid md:grid-cols-2`



`grid grid-cols-1 md:grid-cols-2`



Sin columnas definidas, la columna no se estrecha por debajo del <input>. grid-cols-1 es minmax(0, 1fr): sí puede encogerse. Pruébalo en DevTools a 320 px.

scss/_tokens.scss

```scss
$color-brand-primary: #1e88e5;
$space-lg: 1.5rem;
$radius-md: 0.5rem;
```

=

src/tailwind.css

```css
@theme {
  --color-brand-primary: #1e88e5;
  --spacing-lg: 1.5rem;
  --radius-md: 0.5rem;
}
```

Mismos valores en los dos sitios. Si cambia el azul, cambias las dos líneas y lo apuntas en la guía de estilo.

CLAVE — Si te falta un valor, no lo escribas entre corchetes: añádelo a los dos ficheros de tokens.

¿BEM o Tailwind? Depende del proyecto

	BEM + SASS	Tailwind CSS
Dónde está el aspecto	En el .scss	En el HTML
HTML	Limpio, nombres con significado	Largo, muchas clases
Velocidad para maquetar	Media	Alta
Tamaño del CSS	Crece con cada componente	Pequeño y estable
Tokens	Variables SASS / custom properties	@theme
Encaja mejor con	Webs con HTML escrito a mano, CMS	Proyectos con componentes (React, Vue…)

Y se pueden mezclar

Muchos equipos usan BEM (o componentes) para la estructura y unas pocas utilidades para ajustes puntuales: márgenes entre piezas, ocultar algo en móvil…

Lo que NO funciona es no decidir: medio proyecto con cada sistema y sin norma.

CLAVE — No hay un ganador: hay una decisión de equipo, escrita en la guía de estilo y vigilada por las herramientas.



BLOQUE 4

Validar y documentar

Herramientas que revisan el CSS por ti y una guía de estilo viva (RA2.h, RA2.i)

BLOQUE 4 · VALIDAR (RA2.H)

Tres herramientas, tres preguntas distintas

Validador W3C

¿Es CSS válido según el estándar?

Propiedades que no existen, valores mal escritos, llaves sin cerrar

jigsaw.w3.org/css-validator · a mano

Stylelint

¿Cumple las normas del equipo?

Nombres BEM, ids, !important, colores fuera de los tokens, selectores largos

npm run lint · y subrayado en el editor

Prettier

¿Está bien formateado?

Sangría, espacios, saltos de línea, comillas, una declaración por línea

npm run formato · o al guardar

OJO — El W3C entiende CSS, no SCSS: se le pasa el CSS compilado (F12, o dist/ tras npm run build). Stylelint sí lee el .scss.

BLOQUE 4 · VALIDAR (RA2.H)

Stylelint en el proyecto: ya viene configurado

.stylelintrc.json

```
{
  "extends": "stylelint-config-standard-scss",
  "rules": {
    "selector-class-pattern": [
      "^[a-z][a-z0-9-]*(__[a-z0-9-]+)?(--[a-z0-9-]+)?$",
      { "resolveNestedSelectors": true,
        "message": "Usa BEM: bloque__elemento--mod" } ],
    …  las demás reglas del equipo (siguiente)
  },
  "overrides": [{ "files": ["scss/_tokens.scss"], … }]
}
```

extends

Parte de un conjunto de reglas ya hecho: el estándar, en su versión SCSS

selector-class-pattern

Expresión regular que deben cumplir TODOS los nombres de clase

la regex, en castellano

minúsculas, y opcionalmente __elemento y --modificador. resolveNestedSelectors: revisa también los &__ de SASS

overrides

Excepciones por fichero: en _tokens.scss sí se pueden escribir colores

package.json

```
"scripts": {
  "lint": "stylelint \"scss/**/*.scss\"",
  "arreglar": "stylelint \"scss/**/*.scss\" --fix",
  "formato": "prettier --write \"scss/**/*.scss\" …"
}
```

terminal

```
docker compose run --rm web npm run lint
docker compose run --rm web npm run arreglar
```

CLAVE — Viene en el package.json del kit: no se instala nada. Ya no depende de que alguien se acuerde: la máquina revisa cada fichero.



BLOQUE 4 · VALIDAR (RA2.H)

Las reglas del equipo: qué prohíben y qué haces en su lugar

Regla	No deja pasar	Lo que haces en su lugar
`selector-class-pattern`	.cardTitle · .card__body__title	bloque__elemento--modificador
`selector-max-id: 0`	#menu { … }	una clase (el id, solo en el HTML)
`selector-max-type: 0`	.card h3 · ul li a	una clase BEM (salvo en _base.scss)
`declaration-no-important`	color: red !important	arreglar la especificidad
`color-no-hex · color-named`	#fff · white fuera de _tokens.scss	$color-… (override: en _tokens.scss sí)
`media-feature-name-disallowed-list`	@media (max-width: 768px)	mobile-first: min-width · @include desde()
`max-nesting-depth: 1`	anidar dentro de un anidado	un nivel: &__x, &--x, &:hover
`selector-max-compound-selectors: 2`	.menu .menu__lista .menu__enlace	una clase, o .bloque--mod .bloque__elem
`…-unit-disallowed-list (anchos)`	width: 320px	%, rem, fr, min(): el responsive manda
`length-zero-no-unit`	margin: 0px	margin: 0 (lo arregla npm run arreglar)



NO SE DESACTIVAN — Ni /* stylelint-disable */ ni tocar .stylelintrc.json: el profesor corrige con su copia. Se arregla el código, no la regla.

Leer lo que dice Stylelint

```
docker compose run --rm web npm run lint

scss/components/_card.scss
  12:3  ✖  Usa BEM: bloque__elemento--mod          selector-class-pattern
  18:1  ✖  Nada de #id en el CSS: usa una clase    selector-max-id
  24:14 ✖  Unexpected unit                         length-zero-no-unit

✖ 3 problems (3 errors, 0 warnings)
```

`12:3`

Línea y columna: dónde está el problema (en VS Code, Ctrl+G)

✖ `mensaje`

Qué ha encontrado. Las reglas del equipo lo dicen en castellano

`selector-class-pattern`

El nombre de la regla: búscalo en stylelint.io para saber por qué existe

```
docker compose run --rm web npm run arreglar
```

--fix: arregla lo automático (0px → 0). Los nombres de clase no: los cambias tú, en el SCSS Y en el HTML

TRUCO — Con la extensión de Stylelint en VS Code los errores aparecen subrayados mientras escribes, como las faltas de ortografía.

Prettier: el formato, sin discutirlo

_resenas.scss · antes

```
.resena{padding:$space-md;color:$color-text}
.resena__autor{ font-weight:700 ;}
```

npm run formato

_resenas.scss · después

```
.resena {
  padding: $space-md;
  color: $color-text;
}
.resena__autor {
  font-weight: 700;
}
```

```
{ "printWidth": 120 }
```

.prettierrc.json: líneas de hasta 120 caracteres. El resto, por defecto

terminal

```
docker compose run --rm web npm run formato
# prettier --write "scss/**/*.scss" "*.html" "src/**/*.css"
```

	Prettier	stylelint --fix
Qué toca	Cómo se ve: espacios, saltos, sangría, comillas	Lo que una regla sabe arreglar: 0px → 0
¿Cambia lo que hace el CSS?	Nunca	No
Nombres e #id	No los mira	Los marca, pero no los arregla: tú
Ficheros	SCSS, HTML y CSS	Solo el SCSS

ORDEN — arreglar → formato → lint. Al acabar: lint sin ningún error y el formato pasado. El comprobador mira las dos cosas.

El circuito: que la máquina revise antes que las personas



Por qué importa: en una revisión de código nadie debería perder tiempo discutiendo sangrías o nombres mal escritos. Las herramientas se encargan de lo mecánico y las personas revisan lo que importa: si el componente está bien pensado.

EN LA PRÁCTICA — Antes de cada commit: npm run lint sin errores y npm run formato. La CI con GitHub la veréis en los proyectos.

BLOQUE 4 · GUÍA DE ESTILO (RA2.I)

La guía de estilo: de Penpot al código

Elemento	En el diseño (UD2)	En el código (UD4-UD5)
Colores	Tokens de color en Penpot	$color-brand-primary · --color-brand-primary en @theme
Tipografía	Escala de estilos de texto	Variables de tamaño · clamp() (UD4) · text-xl
Espaciado	Tokens de espaciado	$space-md · --spacing-md
Componentes	Componentes de la librería	Bloques BEM (_card.scss) · catálogo en Storybook (UD7)
Convenciones	—	BEM, Stylelint y cuándo usar utilidades

RA2.i pide UTILIZAR y ACTUALIZAR la guía: si el código cambia y la guía no, la guía miente.

CLAVE — Un token cambiado en Penpot que no llega al código (o al revés) es la divergencia que hay que evitar.

BLOQUE 4 · GUÍA DE ESTILO (RA2.I)

Qué contiene una guía de estilo de código

Tokens

Cada token con su valor y su nombre en SASS y en @theme, lado a lado

Componentes

Cada bloque: para qué sirve, sus elementos, sus modificadores y sus estados (aria-current…)

Convenciones

BEM, el idioma de los nombres, qué no se hace (ids, !important, etiquetas) y cuándo Tailwind

Hacer / no hacer

Ejemplos correctos e incorrectos, como en Material o GOV.UK

Herramientas

Cómo se arranca (docker compose up), cómo se revisa (lint, formato) y con qué reglas

Cambios

Con fecha: «2026-11-02 · Lo que has cambiado». Quien cambia el código, cambia la guía

EN AURIA — Un GUIA-ESTILO.md en la raíz del proyecto, con estas secciones. En la UD7 se convierte en un catálogo con Storybook.



BLOQUE 5

Manos a la obra

Laboratorio AURIA, caso real y errores comunes

LABORATORIO

La tarjeta de AURIA, con dos arquitecturas

1. **Refactor a BEM**
   card y sus __image, __title, __price, __button
2. **BEM en SASS**
   _card.scss con &__ y &--, todo a (0,1,0)
3. **Versión Tailwind**
   La misma tarjeta solo con utilidades, en tailwind.html
4. **Tokens en @theme**
   4 colores y 3 espaciados, con los valores de _tokens.scss
5. **Stylelint y Prettier**
   npm run lint en 0 errores y npm run formato
6. **Guía de estilo**
   GUIA-ESTILO.md: tokens, bloque card y BEM vs Tailwind

Se parte de la tarjeta de la UD4, en el proyecto Docker + Vite. La versión Tailwind es otra página (tailwind.html, con src/tailwind.css y sin SASS) para comparar sin mezclar.

ÉXITO — Las dos tarjetas se ven igual, Stylelint no da errores y cambiar el azul de marca es tocar una sola línea en cada versión.

CASO REAL

Arquitecturas CSS en producción

GOV.UK Frontend

El sistema de diseño del gobierno británico usa BEM con prefijo: .govuk-button--secondary

Tailwind en componentes

Librerías como shadcn/ui o Tailwind Plus reparten componentes hechos solo con utilidades

Linter obligatorio

Los proyectos serios rechazan un cambio si no pasa Stylelint en la integración continua

Tokens como fuente única

Los mismos tokens alimentan Penpot/Figma, el CSS y las apps móviles

Guías públicas

Material Design, GOV.UK o Atlassian publican sus guías con ejemplos de hacer / no hacer

Borrar sin miedo

Con componentes aislados se puede eliminar código muerto sabiendo que no rompe nada

PRUEBA — Abre gov.uk, F12 y mira las clases de cualquier botón: es BEM de libro.

ERRORES COMUNES (1/2)

Trampas de BEM

Nietos en el nombre

Copiar el árbol del HTML en la clase

```
.card__body__footer__button
```

```
.card__button
```

Modificador sin base

Se pierden los estilos comunes

```
<div class="card--destacada">
```

```
<div class="card card--destacada">
```

Volver a anidar

La especificidad sube otra vez

```
.card .card__title { … }
```

```
.card__title { … }
```

Estilar por etiqueta

Cambias h3 por h2 y se rompe

```
.card h3 { … }
```

```
.card__title { … }
```

ERRORES COMUNES (2/2)

Trampas de Tailwind y herramientas

Clases montadas a trozos

Tailwind no las encuentra y no las genera

```
`text-${tono}-600`
```

```
tono === 'ok' ? 'text-green-600' : …
```

Corchetes por todas partes

Valores mágicos otra vez

```
p-[13px] text-[#1e88e5]
```

```
p-md text-brand-primary (@theme)
```

Validar el .scss en el W3C

El validador no entiende SASS

```
Pegar _card.scss en el validador
```

```
Validar el CSS compilado (F12 o dist/)
```

Desactivar la regla que molesta

El linter deja de proteger

```
"selector-class-pattern": null
```

```
Renombrar la clase según BEM
```

CONTROL DE COMPRENSIÓN

Preguntas de control y debate

1. ¿Qué es bloque, qué es elemento y qué es modificador en .card__button--primary?
2. ¿Por qué con BEM no hace falta !important? Explícalo con la especificidad de la UD4.
3. ¿Qué diferencia hay entre escribir .card__title con & en SASS y escribir .card .title?
4. En Tailwind, ¿qué significa md:flex-row y a qué media query de la UD4 equivale?
5. ¿Qué revisa el validador W3C, qué revisa Stylelint y qué hace Prettier?

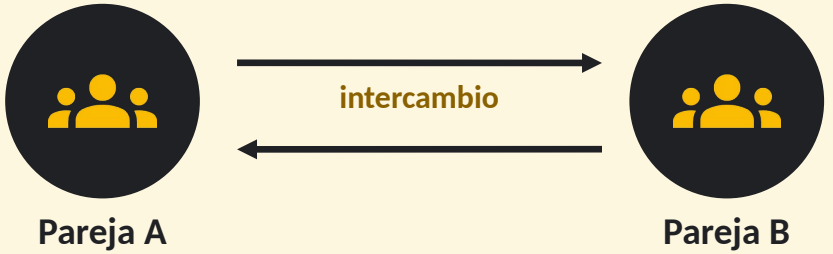
MINI-RETO PRÁCTICO

Reto express: 20 minutos, en parejas

1	**Detectives**	Encontrad 3 webs: una BEM, una Tailwind y una sin convención
2	**BEM**	Renombrad en BEM un componente de la que no tiene
3	**Tailwind**	Rehaced ese componente solo con utilidades
4	**Stylelint**	Pasad vuestro BEM por Stylelint: 0 errores
5	**Veredicto**	¿Cuál elegiríais para ese proyecto y por qué?

Pistas para reconocerlas

F12 → Elements → mirad los atributos class



- Dobles guiones bajos y dobles guiones → BEM
- flex px-4 md:… → Tailwind
- Clases tipo .box2 o ids con estilo → sin convención

ÉXITO — La otra pareja entiende vuestro componente BEM sin abrir el CSS y vuestra elección está justificada.

GLOSARIO · ARQUITECTURA

Arquitectura CSS y BEM

arquitectura CSS	Normas para organizar y nombrar el CSS de un proyecto
BEM	Block, Element, Modifier: convención de nombres de clases
bloque	Componente independiente con sentido propio: .card
elemento	Parte de un bloque, sin sentido fuera de él: .card__title
modificador	Variante de un bloque o elemento: .card--destacada
mezcla (mix)	Un nodo con clases de dos bloques: card__button btn
selector de 2 clases	.bloque--mod .bloque__elem: el modificador del bloque cambia un elemento
estados ARIA	aria-expanded (menú abierto) y aria-current (página actual): el CSS los lee
especificidad plana	Todas las reglas con una sola clase: (0,1,0)
&	En SASS, el selector actual: &__title → .card__title
OOCSS · SMACSS · ITCSS	Otras arquitecturas CSS conocidas
código muerto	CSS que ya no se usa y nadie se atreve a borrar
guía de estilo	Documento vivo con tokens, componentes y normas
convención	Acuerdo de equipo sobre cómo se escribe el código

Tailwind, validación y formato

utility-first	Componer el aspecto con clases de una sola función
clase de utilidad	Clase que aplica una sola declaración: p-4
Tailwind CSS	Framework utility-first; v4 se configura desde el CSS
@tailwindcss/vite	Plugin que hace que Vite compile Tailwind
variante	Prefijo que condiciona una utilidad: hover: md: dark:
@theme	Bloque de Tailwind v4 donde se definen los tokens
valor arbitrario	Valor exacto entre corchetes: w-[327px]
@apply	Junta utilidades dentro de una clase propia
linter	Herramienta que revisa reglas y errores del código
Stylelint	Linter de CSS y SCSS configurable por reglas
Prettier	Formateador automático de código
docker compose run	Con --rm: lanza una orden (npm run …) en el contenedor y lo borra
Validador W3C	Comprueba que el CSS cumple el estándar
CI	Integración continua: revisión automática en el servidor

ÚLTIMA REFLEXIÓN

Dentro de seis meses, ¿quién va a leer tu CSS?

1. **Nombres**

   BEM o utilidades: que cada clase diga lo que es

2. **Tokens**

   Los valores salen de la guía de estilo, no de la cabeza

3. **Herramientas**

   Stylelint y Prettier revisan lo mecánico

Probablemente tú.

Y no te vas a acordar de por qué pusiste .btn-2. Escribe el CSS para esa persona.

Igual que con los tokens: el ahorro no se ve el día que lo escribes, se ve el día que hay que cambiarlo.

DEBATE — ¿Qué haría falta para que un compañero tocara vuestro CSS de AURIA sin preguntaros nada?

CHECKPOINT

Resumen de puntos clave

- Sin convención, el CSS crece, la especificidad sube y nadie se atreve a borrar
- BEM: .bloque__elemento--mod, una clase (0,1,0); el modificador solo trae lo que cambia
- En SASS, &__ y &-- generan nombres BEM sin anidar selectores
- Tailwind (plugin de Vite) compone con utilidades; los tokens entran por @theme
- W3C valida el estándar, Stylelint las normas del equipo, Prettier el formato
- La guía de estilo se usa y se actualiza: si el código cambia y ella no, miente

SIGUIENTE · UD6

Multimedia y rendimiento

- Licencias y derechos de autor
- WebP, AVIF y SVG
- picture, srcset y sizes
- Audio, vídeo y carga diferida

Con la UD5 se cierra el RA2. En la UD6 empezamos el RA3: imágenes, vídeo y audio que pesen poco y carguen rápido.